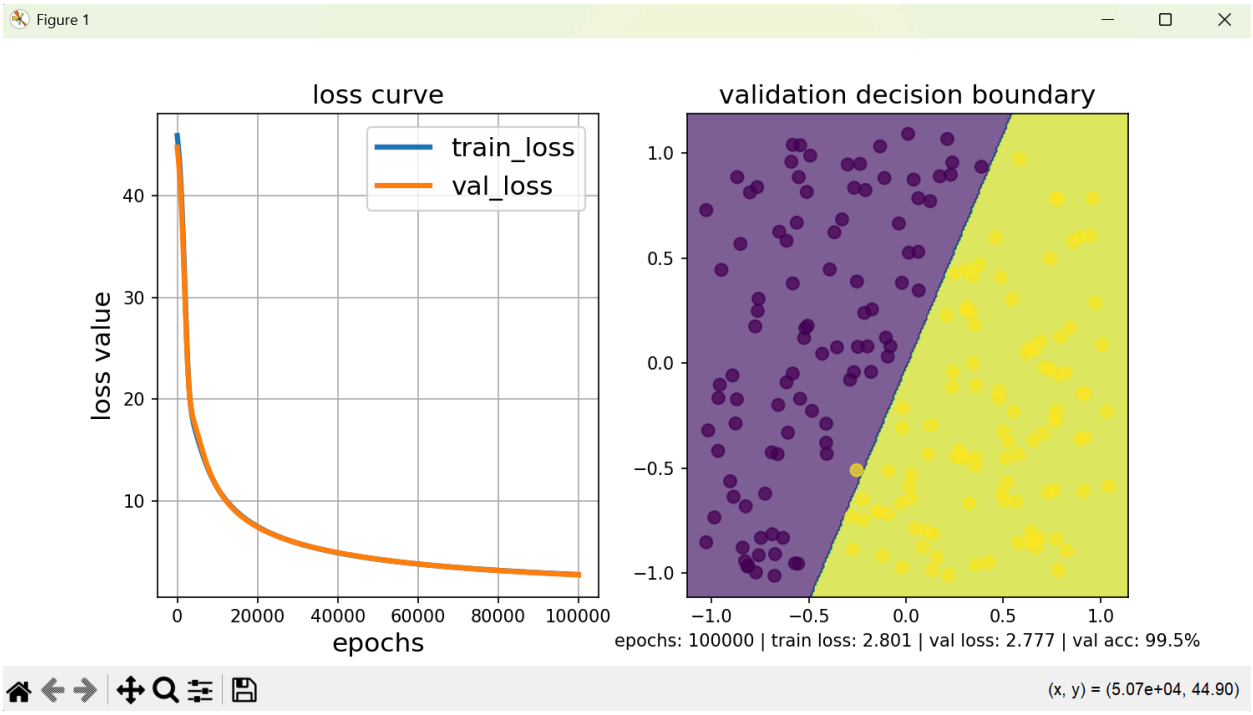
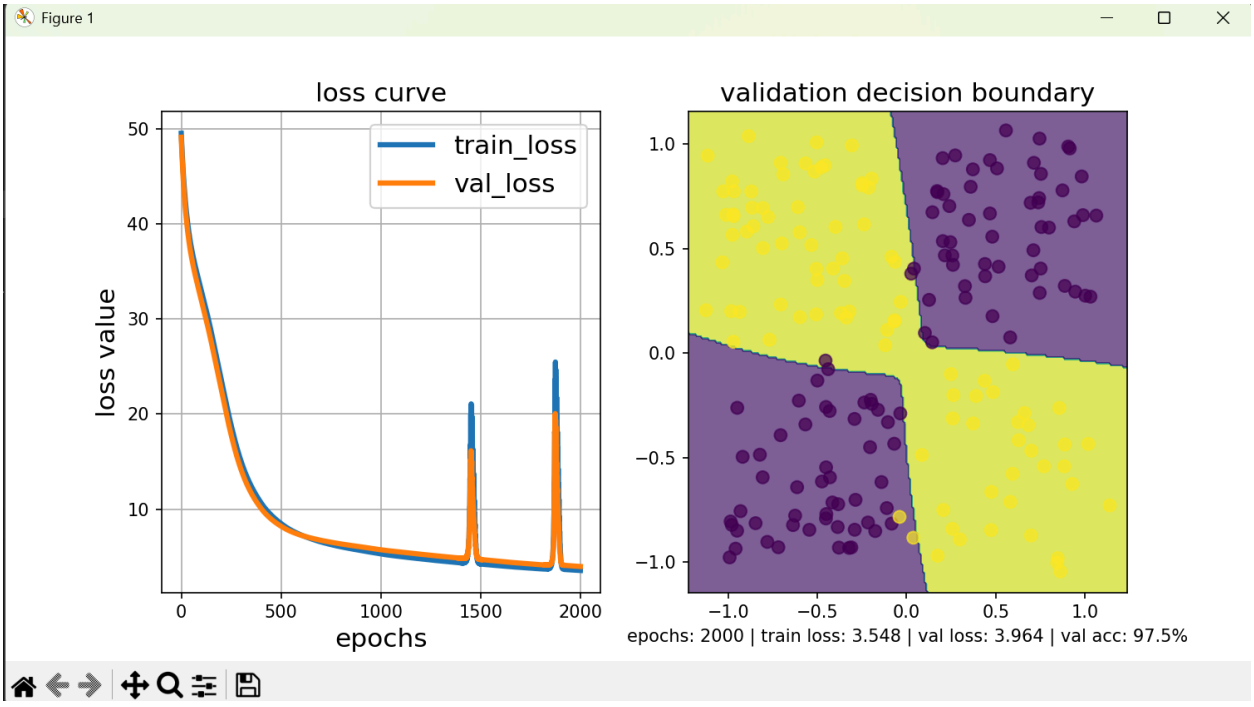
Deliverable 2: Linear Separable Dataset



Deliverable 3: XOR Problem

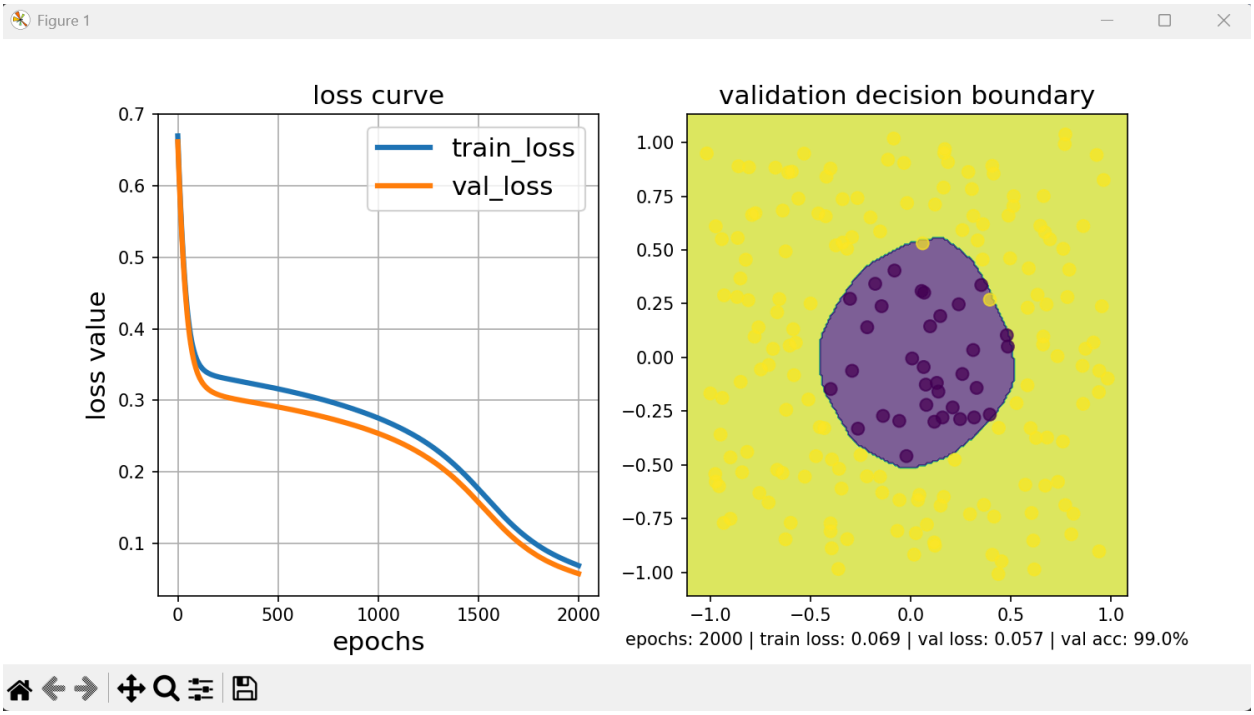


```
hidden_dims = [400, 200]
activations = ['ReLU', 'Tanh', 'Sigmoid']
init = 'Xavier' # 'He' or 'Xavier' is alternative
loss_type = 'L2'

def train(model, X_train, y_train, X_val, y_val, epochs = 2000, lr = 0.001):
```

Because I decided not to use the non-linear embeddings for this question, I wanted something robust. Instead of using x*y as an input, I just used regular x[:, 0] and x[:, 1], but I went for a strong combination of activation functions which I explain later. I opted for Xavier initialization because the dataset is fairly simple, and I knew I didn't need them to change as significantly (compared to for He), so Xavier was the more robust option. I wanted a simple architecture. Due to the simplicity of the data, I opted for L2 loss. I also didn't want many hidden layers because the model would definitely overfit to the data if I chose more. Based on experimentation, I found that starting with a larger number and ending with a smaller number helped, and I thought 400 and 200 were numbers of hidden layers in my network that consistently led to good results. I wanted to leverage the benefits of both ReLU and Tanh. Tanh is good for learning smooth boundaries, so I added this activation function at the end. ReLU helped add non-linearity to the model while enabling the model to be linearly separable, and I think both of these activation functions helped me accomplish a strong XOR approximation equally. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

Deliverable 4: Differences in Cost Function

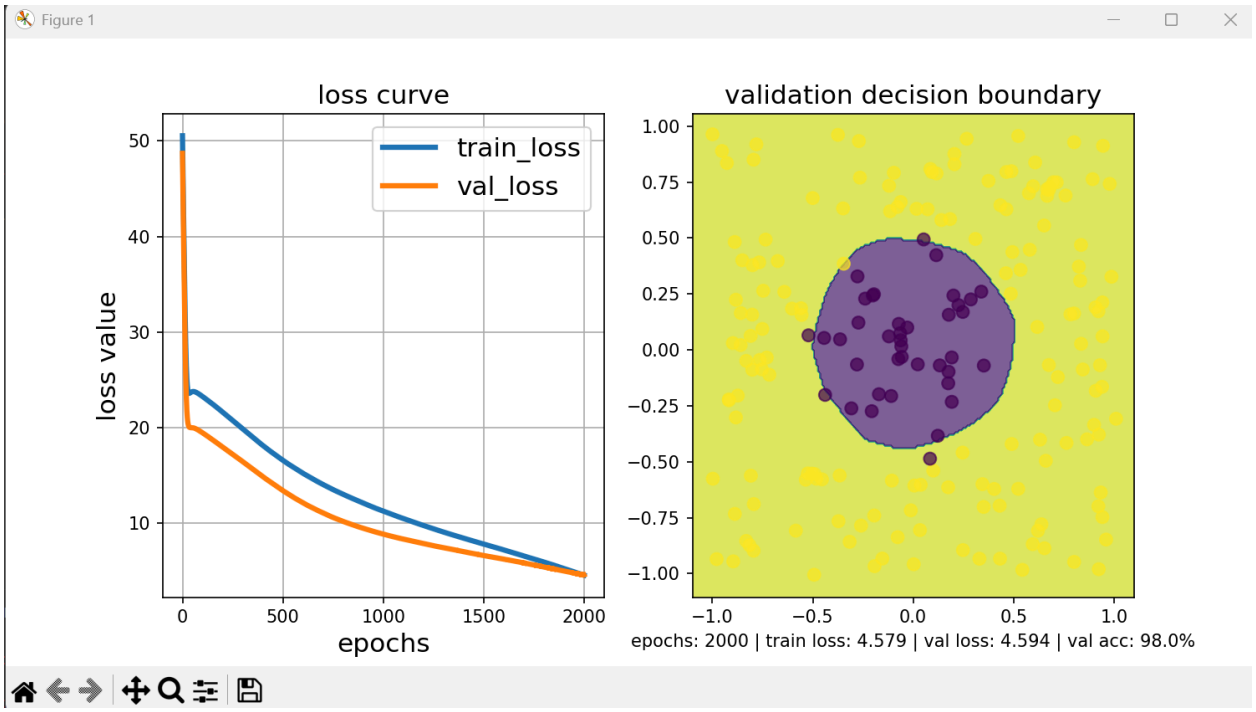
Cross Entropy Loss Model




```
hidden_dims = [200, 200, 500]
activations = ['ReLU', 'Tanh', 'Tanh', 'Sigmoid']
init = 'Xavier' # 'He' or 'Xavier' is alternative
loss_type = 'BCE'
# last layer of model should be sigmoid

def train(model, X_train, y_train, X_val, y_val, epochs = 2000, lr = 0.001):
```

I was required to use the same architecture for both of these models in terms of hidden layers and perceptrons per hidden layer. Because the data is not that complex, I opted for Xavier initialization, since I wanted the model to be more robust. In general, cross entropy loss and tanh activations go well together because CE loss (I used BCE) is stronger with complex decision boundaries compared to L2. Tanh is compatible with CE because of its high nonlinearity. For this reason, I opted for two tanh functions near the end to learn the circle because CE loss depends more on nonlinearity compared to L2 loss. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

L2 Loss Model

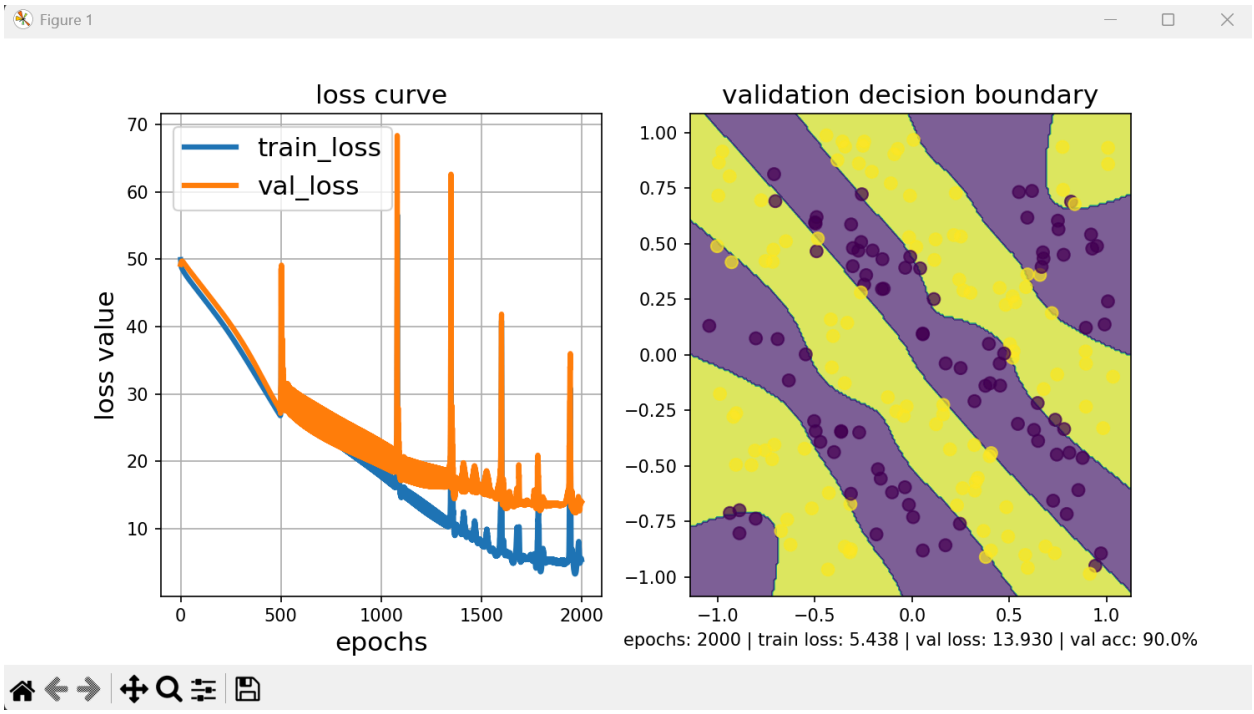



```
hidden_dims = [200, 200, 500]
activations = ['ReLU', 'ReLU', 'Tanh', 'Sigmoid']
init = 'Xavier' # 'He' or 'Xavier' is alternative
loss_type = 'L2'

def train(model, X_train, y_train, X_val, y_val, epochs = 2000, lr = 0.001):
```

In general, L2 loss and ReLU activations go well together because L2 loss is really complementary to linear activations. I decided to go with two ReLU activations this time instead of just one. I tried using the activations that I had for cross entropy for this model, but they just did not perform as well as having two ReLU activations instead of one. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

Deliverable 5: Differences in Optimizers

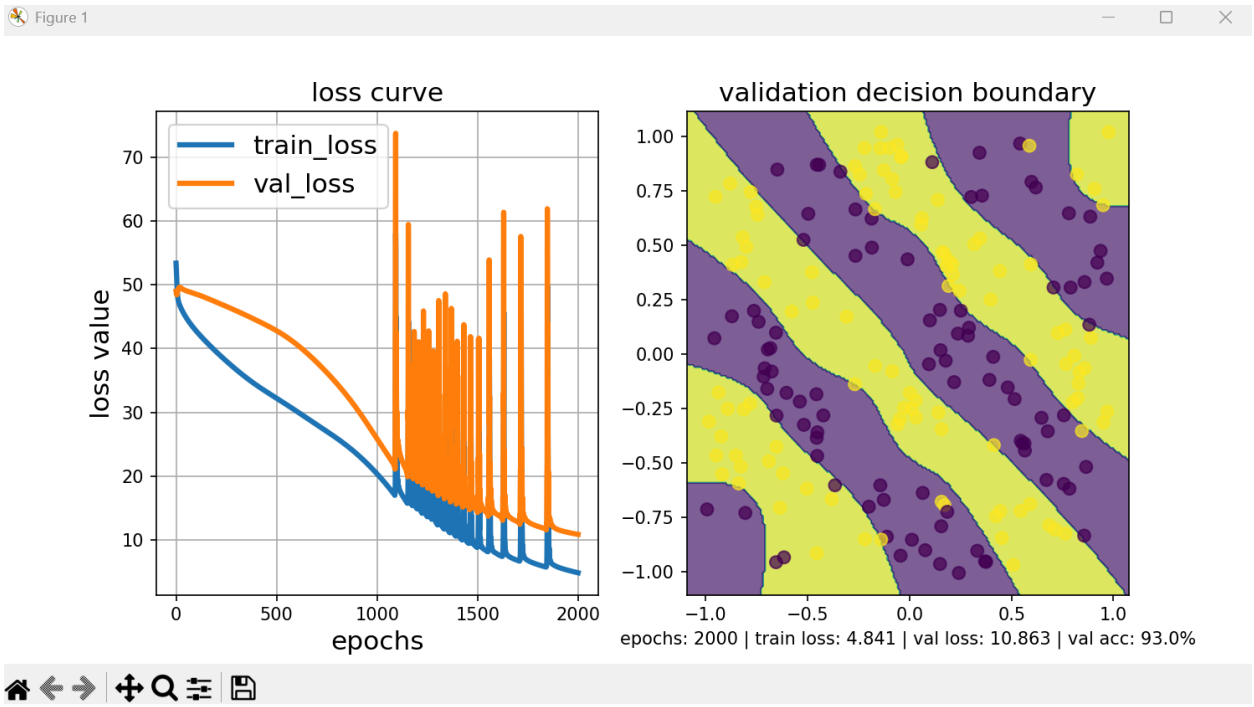
SGD




```
input_dim = np.shape(X)[-1]; output_dim = np.shape(y)[-1]
hidden_dims = hidden_dims = list(map(int, 1 * np.array([200, 100, 100, 100, 100, 100, 100])))
activations = ['Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Sigmoid']
init = 'He' # 'He' or 'Xavier' is alternative
loss_type = 'L2' # primary culprit loss, also num of epochs.

def train(model, X_train, y_train, X_val, y_val, epochs = 2000, lr = 0.001):
```

I used a bunch of tanh activation functions because this was a highly nonlinear problem which required a lot of curved decision boundaries, something tanh accomplishes really well. I used He initialization because I wanted the weights to change really dynamically in order to fit the problem. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

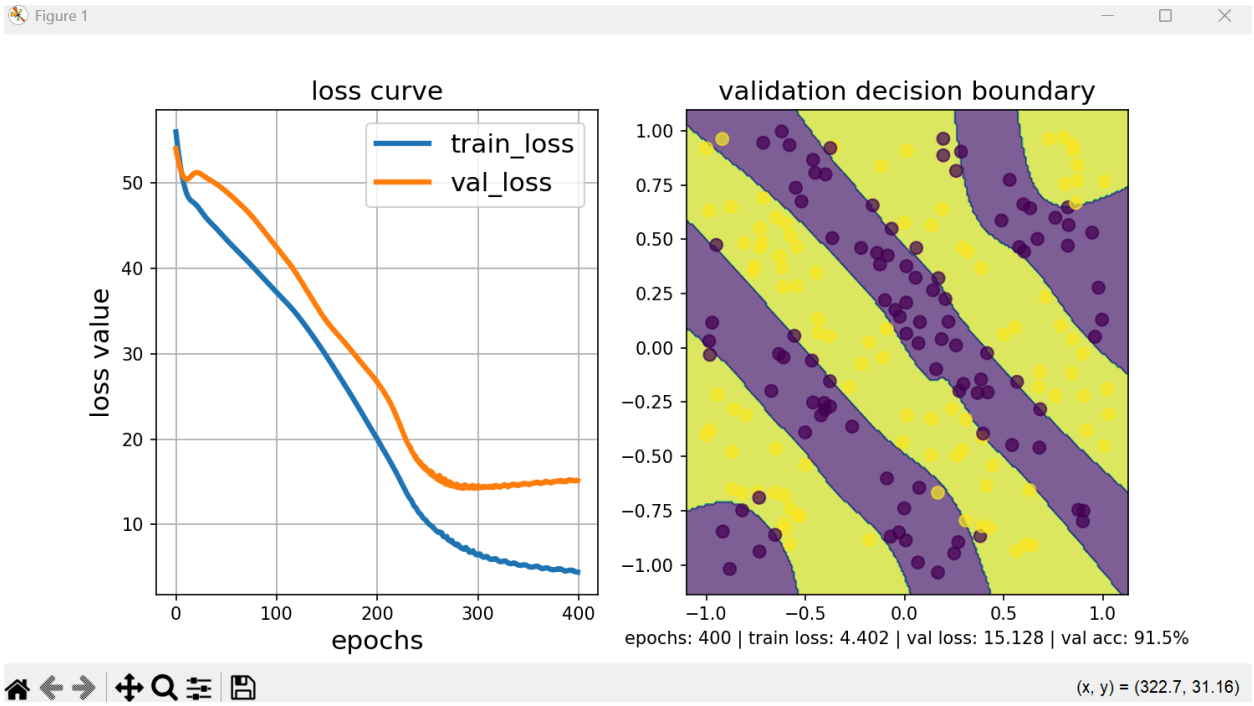
SGD with momentum




```
    hidden_dims = hidden_dims = list(map(int, 1 * np.array([200, 100, 100, 100,
100, 100, 100])))
    activations = ['Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh',
'Sigmoid']
    init = 'He' # 'He' or 'Xavier' is alternative
    loss_type = 'L2' # primary culprit loss, also num of epochs.
    # last layer of model should be sigmoid

    def train(model, X_train, y_train, X_val, y_val, epochs = 2000, lr = 0.0009):

SGD_W_MOMENTUM(momentum_term = 0.6)
```

I used a momentum term of 0.6 because I knew that I already had a good base model, so I didn't want to accelerate too fast. Similar to the SGD model, I used a bunch of tanh activation functions because this was a highly nonlinear problem which required a lot of curved decision boundaries, something tanh accomplishes really well. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

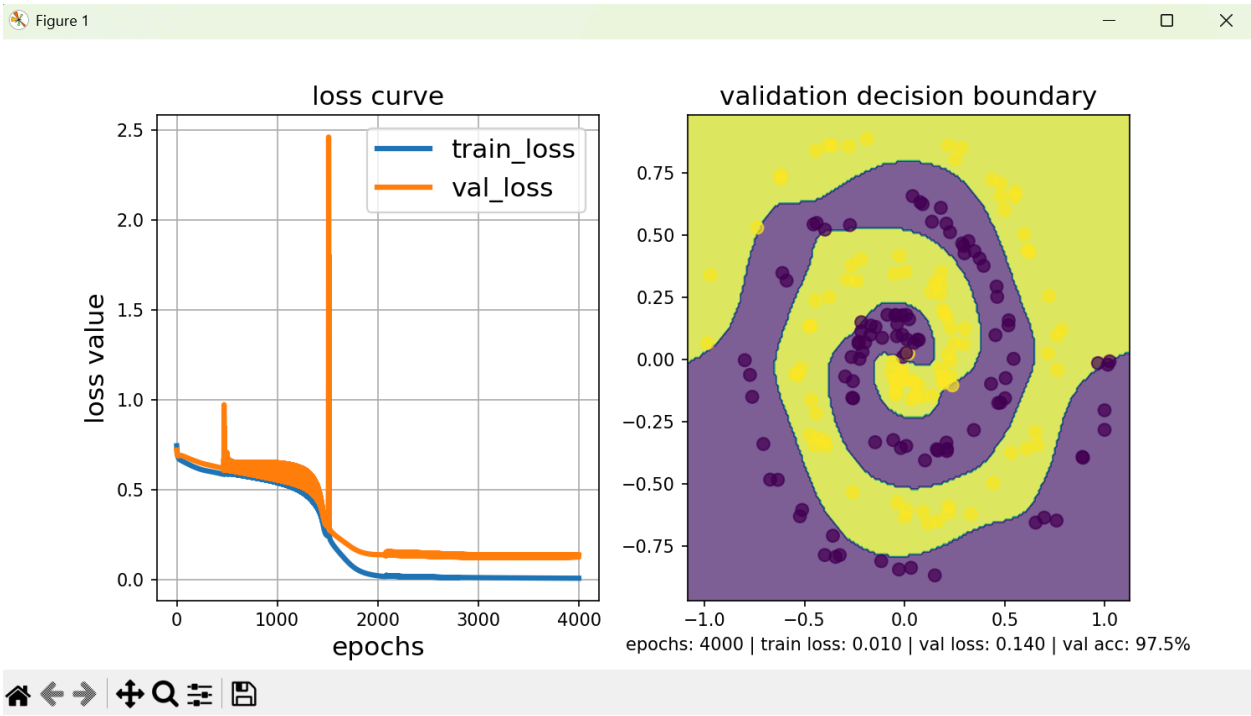
ADAM




```
hidden_dims = hidden_dims = list(map(int, 0.5 * np.array([200, 100, 100, 100, 100, 100])))
activations = ['Tanh', 'Tanh', 'Tanh', 'Tanh', 'Tanh', 'Sigmoid']
init = 'He' # 'He' or 'Xavier' is alternative
loss_type = 'L2' # primary culprit loss, also num of epochs.
# last layer of model should be sigmoid

def train(model, X_train, y_train, X_val, y_val, epochs = 400, lr = 0.0009):


ADAM(beta_1 = 0.90, beta_2 = 0.999) # increasing beta_1 from 0.9 might help
```

While the ADAM method has a lower accuracy compared to the SGD with momentum, it also has far fewer epochs. The ADAM method really shined in terms of its ability to quickly converge while maintaining high accuracy. Because the ADAM optimizer has different learning rates for all the weights and biases, the model converged much quicker, and was able to mostly learn the sinusoidal data from just 400 epochs. I used the default input and output layers, including the sigmoid activation function for proper classification of the data.

Deliverable 6: Swiss Roll:



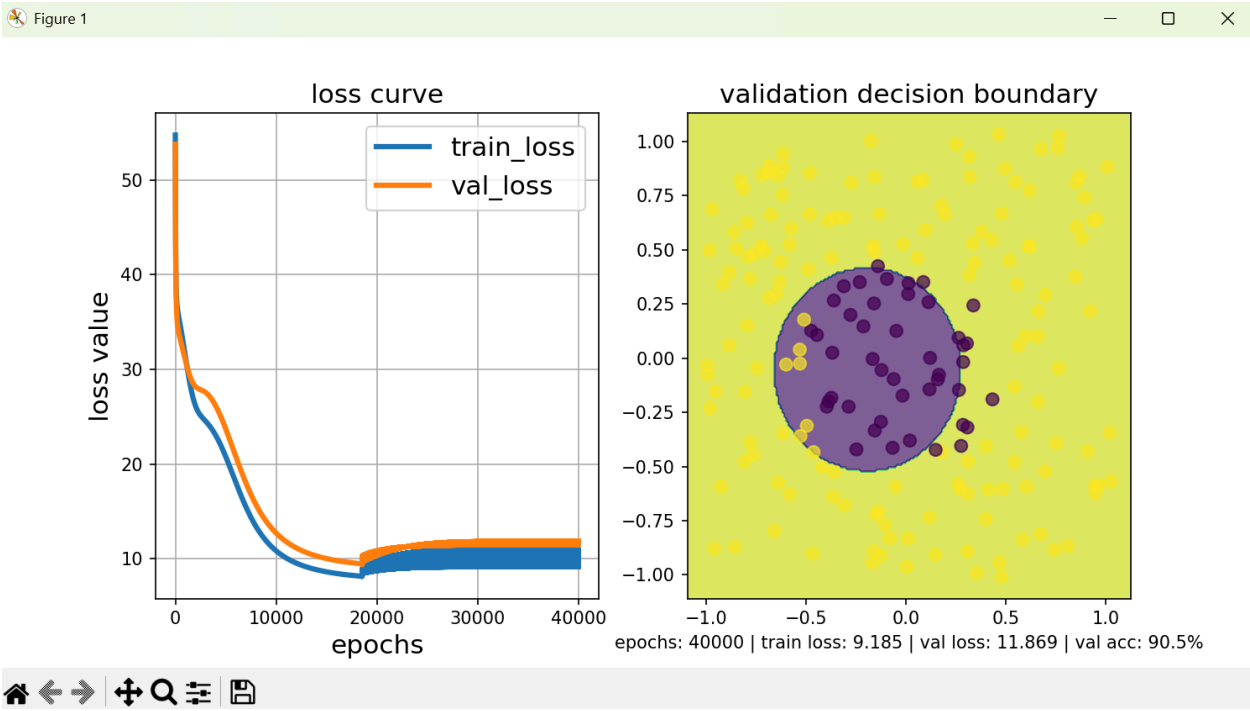

```
hidden_dims = list(map(int, 2.2 * np.array([125, 100, 100, 75])))
activations = ['Tanh', 'Tanh', 'Tanh', 'Tanh', 'Sigmoid']
init = 'He' # 'He' or 'Xavier' is alternative
loss_type = 'BCE' # primary culprit loss, also num of epochs

def train(model, X_train, y_train, X_val, y_val, epochs = 4000, lr = 0.005):
```

For this problem, I used ‘He’ initialization because I knew I wanted a lot more variance in my weights. I knew that they would need to be tuned a lot more than normal due to the insane non-linearity of this problem. I also knew I wanted plenty of tanh functions and that I would default to BCE loss, since this problem is highly non-linear and requires an incredibly complex non-linear decision boundary. For this reason, I exclusively used tanh activation functions. I used several hidden layers because I knew that I would need to expose the model to a lot of nonlinearity for it to manually learn the swiss roll.

I knew that I wanted to start with many perceptrons per layer, but I really wanted to shrink the number of perceptrons throughout the forward pass so the model would pick up on higher-level features. However, I used this list(map(...)) idea in order to keep the relative weights while still changing the amount of potential overfitting/underfitting present in the model.

Deliverable 7: Non-Linear Embeddings:

Circle from non-linear embedding:



non-linear embeddings: (i just added x2 + y **2)**

```
    x2_y2_train = (X_train[:,0] ** 2 + X_train[:,1] ** 2).reshape(-1,1)
    x2_y2_val = (X_val[:,0] ** 2 + X_val[:,1] ** 2).reshape(-1,1)
```

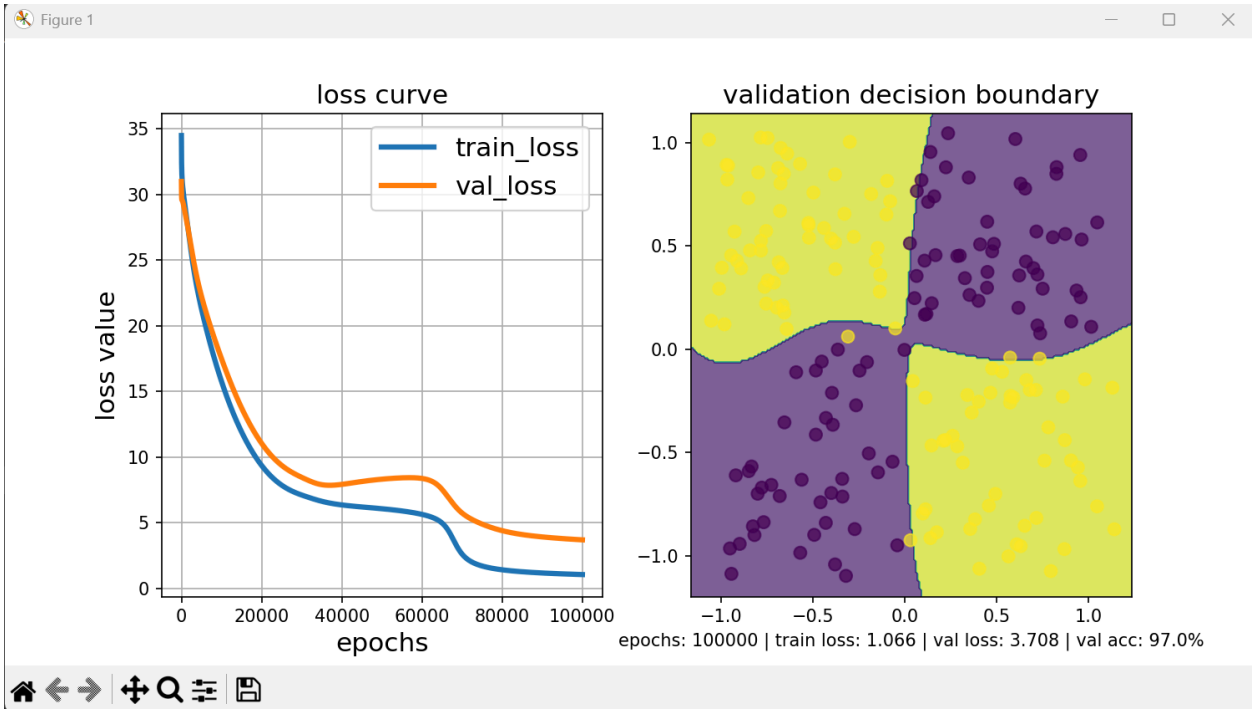
model architecture:

```
    hidden_dims = [500, 100]
    activations = ['Linear', 'Linear', 'Sigmoid']
    init = 'Xavier' # 'He' or 'Xavier' is alternative
    loss_type = 'L2' # primary culprit loss, also num of epochs.
    # last layer of model should be sigmoid
```

train params:

```
    def train(model, X_train, y_train, X_val, y_val, epochs = 40000, lr =
0.0001):
```

XOR from non-linear embedding:



I used x*y as a non-linear embedding

non-linear embeddings:

```
  xy_train = (X_train[:,0] * X_train[:,1]).reshape(-1,1)
    xy_val = (X_val[:,0] * X_val[:,1]).reshape(-1,1)
```

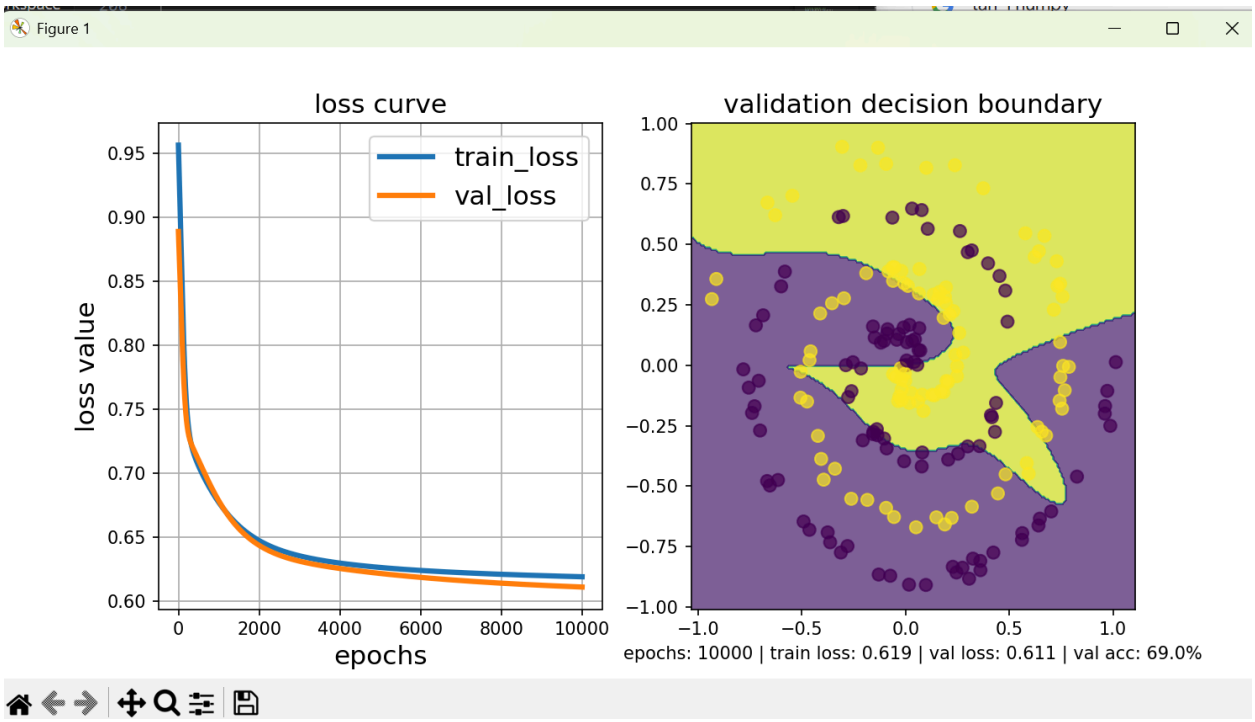
architecture:

```
hidden_dims = [4, 2] # [2, 1] works, too, but it's less consistent.
    activations = ['Tanh', 'Linear', 'Sigmoid'] # Keep last layer linear for
regression and classification
    init = 'He' # 'He' or 'Xavier' is alternative
    loss_type = 'L2' # primary culprit loss, also num of epochs.
    # last layer of model should be sigmoid
```

training params:

```
    def train(model, X_train, y_train, X_val, y_val, epochs = 100000, lr =
0.0001):
```

Swiss Roll from non-linear embedding:



Honestly, I really struggled to get this model to work. I tried many things, including adding a second nonlinear term, arctan. I am mostly satisfied with how the model was able to learn the little roll in the middle. I would've spent more time with this, but I had already spent over an hour on just this model.

added non-linearity terms: (sqrt(x**2 + y**2) alongside arctan)

```
sqx2_y2_train = (np.sqrt(X_train[:,0] **2 + X_train[:,1] ** 2)).reshape(-1,1)
 sqx2_y2_val =  (np.sqrt(X_val[:,0] **2 + X_val[:,1] ** 2)).reshape(-1,1)
 arctan_train = (np.arctan2(X_train[:,1], X_train[:,0])).reshape(-1,1)
 arctan_val =  (np.arctan2(X_val[:,1], X_val[:,0])).reshape(-1,1)
```

architecture:

```
 hidden_dims = list(map(int, 0.2 * np.array([125, 75])))
 activations = ['ReLU', 'Tanh', 'Sigmoid']
 # activations = ['Tanh', 'Linear', 'Sigmoid']

 init = 'He' # 'He' or 'Xavier' is alternative
 loss_type = 'BCE'
```

training params:

```
    def train(model, X_train, y_train, X_val, y_val, epochs = 10000, lr =
0.0001):
```